

Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' QA-VALIDACION (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: Página de Saludos Internacionales Tsoft (XP-12)

1. CONTEXTO

Se requiere una página HTML/CSS estática que muestre saludos coloquiales de todos los países donde Tsoft tiene oficinas. El requerimiento es explícitamente 'una página en HTML y CSS'. No hay backend mencionado, no hay autenticación de usuarios, no hay base de datos dinámica, no hay API. El entorno de ejecución primario es LOCAL (protocolo file:///), lo que activa la restricción crítica de arquitectura de archivo único o mínima fragmentación sin módulos ES6 nativos.

2. DECISIÓN ARQUITECTÓNICA

Arquitectura elegida: Static Single-File Application (SSFA)

- Un archivo `index.html` que contiene toda la estructura semántica.
- Un archivo `styles.css` para separación limpia de presentación (permitido y recomendado incluso en entorno local).
- Un archivo `main.js` (vanilla JS plano, SIN type='module', SIN import/export) para interactividad mínima: filtro de búsqueda por país y sanitización de inputs.
- CERO dependencias de framework (React, Vue, Angular: PROHIBIDOS por YAGNI).
- CERO backend, CERO base de datos, CERO servidor de aplicaciones.
- CERO archivos de imagen binarios: se usarán placeholders via placeholder.co y texto SVG inline para el logo.

3. JUSTIFICACIÓN BASADA EN YAGNI / KISS / BORING TECHNOLOGY

- **¿Por qué NO microservicios?** El requerimiento es una página informativa estática. No hay equipos independientes, no hay despliegue separado de componentes. Microservicios añadirían 10x la complejidad operacional con 0x el beneficio.
- **¿Por qué NO React/Vue/Next.js?** YAGNI puro. Una lista de saludos con filtro no justifica un bundler, un virtual DOM, ni un proceso de build. El tiempo de setup superaría el tiempo de desarrollo real.
- **¿Por qué NO base de datos?** Los datos (saludos por país) son estáticos y conocidos. Un array JavaScript en el propio archivo es la solución más simple, más rápida y más mantenible. PostgreSQL para 30 registros estáticos sería ingeniería absurda.
- **¿Por qué NO type='module'?** El entorno es LOCAL (file:///). Los navegadores bloquean módulos ES6 bajo protocolo file:// por política CORS. Un solo archivo JS plano elimina este riesgo completamente.
- **¿Por qué NO CDN de Bootstrap/Tailwind?** Requiere conexión a internet. Una página corporativa interna debe funcionar offline. CSS custom con variables es suficiente y más controlable.

4. MANEJO DE SEGURIDAD (Adaptado a contexto estático)

- **XSS**: El único vector de ataque real es el campo de búsqueda/filtro. Se implementará sanitización via `textContent` en lugar de `innerHTML` al renderizar resultados dinámicos. NUNCA se usará `innerHTML` con input del usuario.
- **HTTPS**: Recomendación documentada en comentarios del código para despliegue en servidor. En entorno local no aplica, pero el código está preparado para servirse bajo HTTPS sin modificaciones.
- **Controles de acceso**: No aplica en MVP estático. El contenido es público por naturaleza. Se documenta como deuda técnica si se requiere CMS en el futuro.
- **SQL Injection**: No aplica. No hay base de datos ni queries.

5. RIESGOS ASUMIDOS POR SIMPLIFICAR

- **Riesgo 1 (BAJO)**: Si en el futuro se requiere edición dinámica del contenido por usuarios no técnicos, se necesitará migrar a un CMS o backend. Mitigación: los datos están centralizados en un array JS fácilmente exportable a JSON.
- **Riesgo 2 (BAJO)**: Si la lista de países crece a 100+, el filtro JS en memoria seguirá siendo performante ($O(n)$ sobre array pequeño). No hay riesgo de performance.
- **Riesgo 3 (MEDIO)**: Las medidas de seguridad avanzadas (autenticación, RBAC) no están implementadas porque el requerimiento no define usuarios con roles. Si se añaden en el futuro, requerirá arquitectura diferente.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- main.js | Estado: APROBADO | Iteraciones: 1
- styles.css | Estado: APROBADO | Iteraciones: 1
- README.md | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 39,272

Tokens de salida (output): 12,903

Total tokens: 52,175

Modelo utilizado: claude-sonnet-4-6

Ø=Üh Ø=Ü» ~12.9 horas-hombre ahorradas

"H squad de 4 personas × 1.7 días

Desglose por Agente:

DISEÑO-ARQUITECTURA: !"2,343 input | !"1,348 output | 1 llamadas | ~1.7h ahorradas

MICROPLANIFICACION-DISEÑO: !"9,888 input | !"6,000 output | 4 llamadas | ~5.3h ahorradas

FRONTEND-DESARROLLO: !"10,265 input | !"5,407 output | 4 llamadas | ~3.3h ahorradas

QA-VALIDACION: !"16,776 input | !"148 output | 4 llamadas | ~2.7h ahorradas